Market Making with Deep Reinforcement Learning from Limit Order Books

Reproduction, Data Adaptation, and Robustness Extensions

Jiaan Han Xiangpeng Huang Tianle Liu

Abstract

This project studies a deep reinforcement learning approach to market making from limit order book data. We run experiments on two market data sources: LOBSTER NASDAQ order book data for AAPL, AMZN, and MSFT, and Shenzhen Stock Exchange Level-2 data for 10 selected Chinese stocks. We begin from the paper *Market Making with Deep Reinforcement Learning from Limit Order Books*, which uses an Attn-LOB representation model before training a reinforcement learning market maker. We reimplemented the pipeline in PyTorch, added reusable training and evaluation interfaces, reconstructed a Shenzhen Level-2 data set from order and trade events, and compared reinforcement learning agents with fixed-level, random, and Avellaneda-Stoikov baselines. We also added robustness experiments that separate observation latency, order-arrival latency, order loss, and model inference latency. Our strongest classification result is a joint 10-stock Attn-LOB encoder with test macro F1 of 0.701 on Shenzhen data. In trading backtests, a simple fixed-level baseline is strongest on the Shenzhen sample, while the joint Dueling-DQN agent is strongest on the three-stock LOBSTER sample. These results suggest that the learned representation is useful, but that profitable learned market making remains highly sensitive to the data source, execution model, latency assumptions, and inventory control.

1 Introduction

Market making is the problem of continuously posting bid and ask quotes, earning spread revenue while managing inventory and adverse selection risk. In this project, we evaluate this problem on both LOBSTER NASDAQ data and Shenzhen Stock Exchange Level-2 data, so that the same modeling pipeline can be tested on the original-style U.S. order book format and on a reconstructed Chinese limit order book data set. The task is well suited to reinforcement learning (RL) because actions affect future inventory, realized PnL, and risk exposure. It is also difficult: the agent observes a high-dimensional, rapidly changing limit order book (LOB), while executions depend on queue position, latency, liquidity, and short-horizon price movement.

The original paper we studied proposes a two-stage architecture. First, an Attn-LOB neural network is trained on limit order book states to predict short-horizon mid-price movement. Second, the learned encoder is used inside a market-making reinforcement learning agent. Our project reproduces this idea and extends it in four directions:

1. We rewrote the Attn-LOB model and training loop in PyTorch.
2. We built a full data pipeline for Shenzhen Level-2 order and trade data, reconstructing an event-level 10-level order book.
3. We evaluated PPO and Dueling-DQN against interpretable non-learning baselines.

4. We added robustness tests for observation latency, order latency, order loss, and inference latency, plus regime-sensitivity analysis.

All final experiment outputs are stored locally under `Our-Code/experiment_results/`. The code interface is documented in `Our-Code/README.md`, `Our-Code/SYSTEM_README.md`, and `Our-Code/PROJECT_RUNBOOK.md`.

2 Original Paper and Reproduction Target

The original paper combines representation learning and reinforcement learning. The representation stage uses a convolutional and attention-based LOB model to predict whether the future mid-price moves down, stays stationary, or moves up. This supervised task is not the final trading objective, but it encourages the encoder to extract short-horizon price formation signals from the LOB. The RL stage then uses that encoder to select market-making quotes.

Our first reproduction target was a one-day LOBSTER AAPL experiment. During the initial model search, we compared the original Attn-LOB configuration with wider architectures and different optimization choices. Table 1 summarizes the most relevant early pretraining results from our project slides. The best modified setting used AdamW and weighted cross entropy with embedding, convolution, and inception dimensions of 96/32/96.

Table 1: Initial AAPL Attn-LOB tuning results from the reproduction stage.

Setting	Optimizer + loss	Size	Params	Precision	Recall	F1
Original	Adam + CE	64/32/64	271k	0.438	0.454	0.434
Best architecture-only	Adam + CE	128/48/128	613k	0.461	0.458	0.453
Best modified	AdamW + weighted CE	96/32/96	377k	0.505	0.518	0.510

We used this best modified Attn-LOB configuration for the larger experiments in this report.

3 Data Sets

3.1 LOBSTER Sample Data

The first data source is a LOBSTER sample containing AAPL, AMZN, and MSFT for June 21, 2012. LOBSTER already provides message files and 10-level order book snapshots. We convert the files into a common event format used by both pretraining and RL. The sample is useful for reproducing the original data format, but it is small: each stock has only one trading day.

3.2 Shenzhen Level-2 Data

The second data source is a five-day Shenzhen Level-2 data set covering January 21, 22, 23, 24, and 27, 2025. Unlike LOBSTER, the uploaded data include separate market snapshot, order, and trade HDF5 files. We therefore built a replay pipeline:

1. read order submissions and trade/cancel records;
2. maintain active buy and sell order maps by order id;

3. aggregate active orders into price levels;

4. emit 10-level LOB states and event features;

5. validate reconstructed best bid/ask against market snapshots;

6. compute train/validation/test labels from future mid-price movement.

We selected 10 Shenzhen stocks using only data-quality criteria: completeness across dates, activity, spread, valid-book fraction, label balance, and non-stationary signal. The final split is date based: January 21–23 for training, January 24 for validation, and January 27 for testing. This prevents future-day leakage. The processed event files contain LOB depth, execution features, split id, and labels in $\{-1, 0, 1\}$.

Table 2: Main experiment data sets.

Data set	Securities	Dates	Split	Main use
SZ10	10 Shenzhen stocks	5 days	3:1:1 by date	Main large-scale experiment
LOBSTER3	AAPL, AMZN, MSFT	1 day	intraday split	Original-format comparison

4 Methodology

4.1 Attn-LOB Pretraining

The supervised pretraining target is the three-class future mid-price movement label. Given a 50-event LOB window, the model predicts down, stationary, or up. The encoder contains convolutional feature extraction across price/size columns, inception-style temporal filters, and multi-head attention. We train either:

- a **single-stock encoder**, trained separately for each stock; or
- a **joint encoder**, trained once on all selected stocks and reused by per-stock RL agents.

Joint pretraining is meant to test whether shared order-book structure transfers across securities.

4.2 Reinforcement Learning Environment

The RL environment is an event-driven market-making simulator. At each decision step, the agent observes normalized LOB windows, auxiliary event features, and inventory state. The environment tracks cash, inventory, mark-to-market value, trading volume, average spread, realized PnL, normalized PnL, PnL per map, and inventory penalties.

We evaluate two agent families:

- **PPO**, using a continuous action that controls reservation price bias and quote spread.
- **Dueling-DQN**, using a discrete action space for quote placement and inventory management.

The reward combines dampened PnL, trading PnL, and an inventory penalty. All reported results use zero explicit transaction cost and zero market impact, which matches our small-order, high-frequency market-making assumption. The code still contains transaction-cost parameters for future sensitivity checks.

4.3 Baselines

We compare the RL agents with non-learning quote policies:

- fixed level 1, 2, or 3: quote at the corresponding LOB level;
- random top-5-level quoting: sample bid and ask levels independently;
- Avellaneda-Stoikov: quote around an inventory-adjusted reservation price.

These baselines are important because raw RL PnL is difficult to interpret without a simple market-making benchmark.

4.4 Latency, Order Loss, and Regime Tests

The original paper-style latency is an observation delay: the agent observes state $t-\ell$ when quoting at event t. We extended this into several execution-friction modes:

- observation latency in events;
- order-arrival latency in events or milliseconds;
- independent bid/ask order loss;
- simulated inference latency added to order arrival;
- measured model decision time.

We also group test episodes into low, mid, and high regimes by realized volatility, average spread, and trade intensity.

5 Implementation

The implementation is organized as a reusable system rather than a collection of one-off scripts. The main entry point is:

```
cd Our-Code
../.venv/bin/python3 run_system.py <command>
```

The full SZ10 pipeline can be run or resumed with:

```
../.venv/bin/python3 run_system.py full --stage all
```

Individual stages include `pretrain`, `rl`, `baseline`, `latency`, `regime`, `summarize`, and `plots`. The LOBSTER3 comparison can be reproduced with:

```
../.venv/bin/python3 run_lobster3_full_experiment.py --stage all
```

The data preparation scripts are grouped under `Our-Code/data_prep/`. The API wrapper is in `Our-Code/system_api.py`.

6 Results

6.1 Pretraining Results

Table 3 shows the main classification results. Joint pretraining is especially strong on the Shenzhen data, reaching 0.701 test macro F1. On LOBSTER3, the joint encoder also improves macro F1 relative to the single-stock encoders.

Table 3: Attn-LOB pretraining results.

Data	Scope	Accuracy	Precision	Recall	F1
SZ10	Best single stock (002193.SZ)	0.725	0.655	0.668	0.658
SZ10	Joint 10-stock encoder	0.706	0.702	0.701	0.701
LOBSTER3	Best single stock (MSFT)	1.000	0.333	0.333	0.333
LOBSTER3	Joint 3-stock encoder	0.658	0.485	0.692	0.497

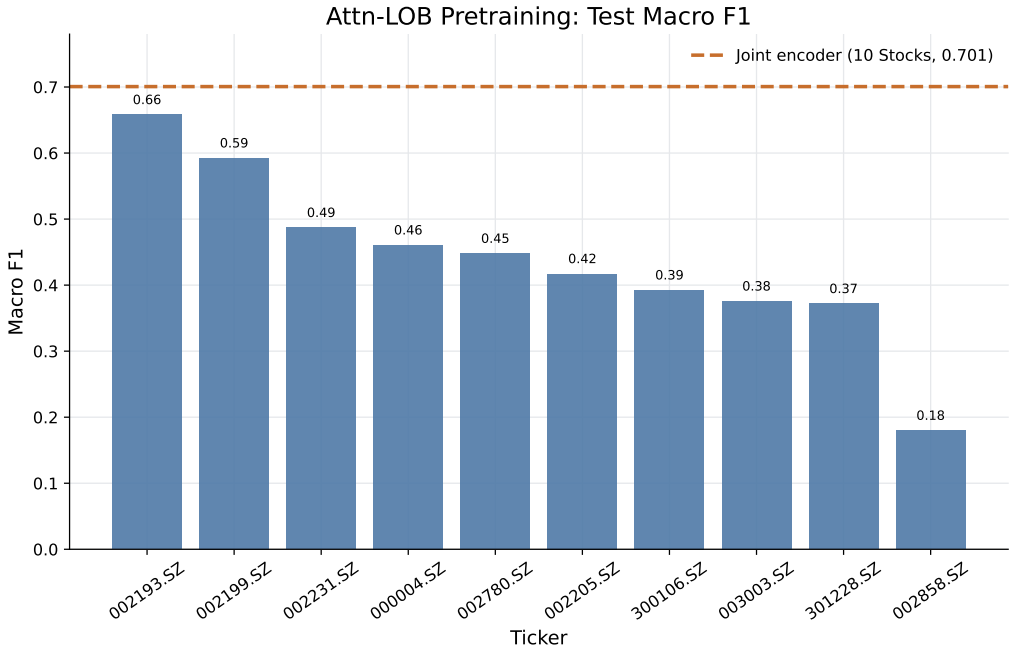


Figure 1: SZ10 pretraining F1 comparison for single-stock and joint encoders.

6.2 Trading Results on SZ10

Table 4 reports test metrics averaged across the 10 selected Shenzhen stocks. Fixed level 2 is the best method by mean PnL and Sharpe-like stability, followed by fixed level 3 and joint PPO. This is an important result: although the joint encoder predicts price movement well, the learned RL policy does not dominate simple spread-capture baselines on this short sample.

Table 4: SZ10 aggregate trading results on the test day.

Method	PnL	ND-PnL	PnL-MAP	Avg abs inv.	Volume
baseline_fixed_2	13.97	406.36	0.110	236.60	9,198
baseline_fixed_3	6.57	103.83	0.069	91.69	2,424
joint_ppo	3.50	17.63	0.081	152.72	9,715
baseline_as	0.54	4.61	0.027	50.79	1,412
baseline_random_5level	0.09	9.25	0.016	314.66	17,071
single_ppo	-2.22	-48.30	-0.020	142.42	13,189
joint_dueling_dqn	-8.83	-258.03	-0.020	240.29	32,308
single_dueling_dqn	-9.85	-279.50	-0.028	269.65	31,059
baseline_fixed_1	-20.62	-1355.77	-0.021	508.06	61,707

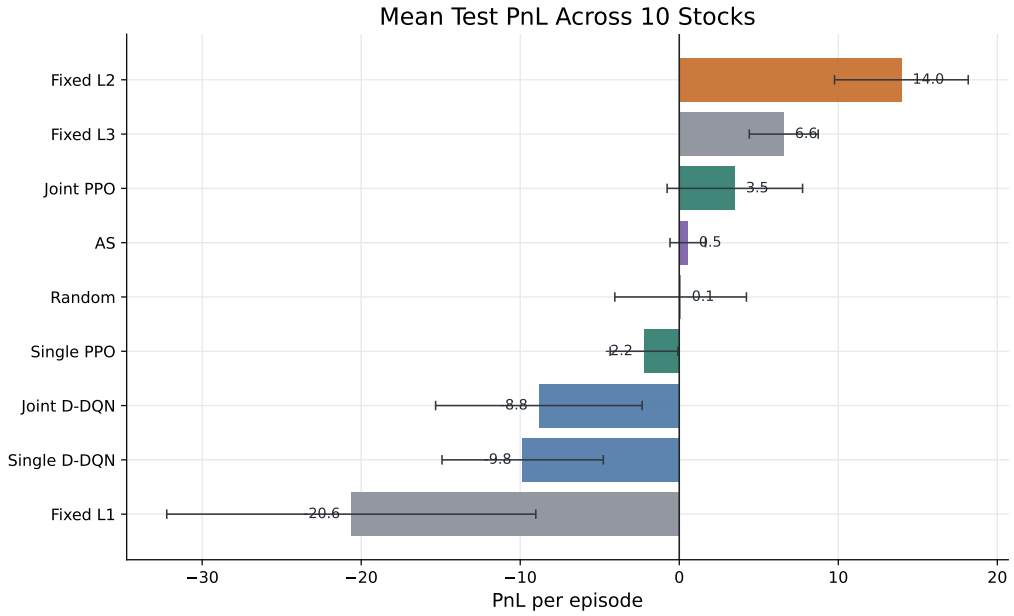


Figure 2: SZ10 overall test PnL comparison.



Figure 3: SZ10 risk, return, and inventory diagnostics.

6.3 Trading Results on LOBSTER3

Table 5 reports the AAPL/AMZN/MSFT comparison. Here the ranking changes: joint Dueling-DQN has the best mean PnL, while PPO performs poorly. The difference between SZ10 and LOBSTER3 shows that agent rankings are data-dependent and that a policy that is viable in one market sample may not generalize automatically.

Table 5: LOBSTER3 aggregate trading results on the test split.

Method	PnL	ND-PnL	PnL-MAP	Avg abs inv.	Volume
joint_dueling_dqn	11.03	46.08	0.026	233.72	288,218
baseline_fixed_2	0.00	0.00	0.000	0.00	0
baseline_fixed_3	0.00	0.00	0.000	0.00	0
baseline_random_5level	-2.67	-23.96	0.021	123.35	122,066
baseline_fixed_1	-9.70	-560.94	0.003	464.16	481,241
baseline_as	-11.77	-114.19	0.006	404.00	526,865
single_dueling_dqn	-30.60	-322.56	-0.061	333.30	277,048
joint_ppo	-79.57	-1098.72	-0.121	406.74	844,747
single_ppo	-99.07	-1692.37	-0.149	406.73	900,857

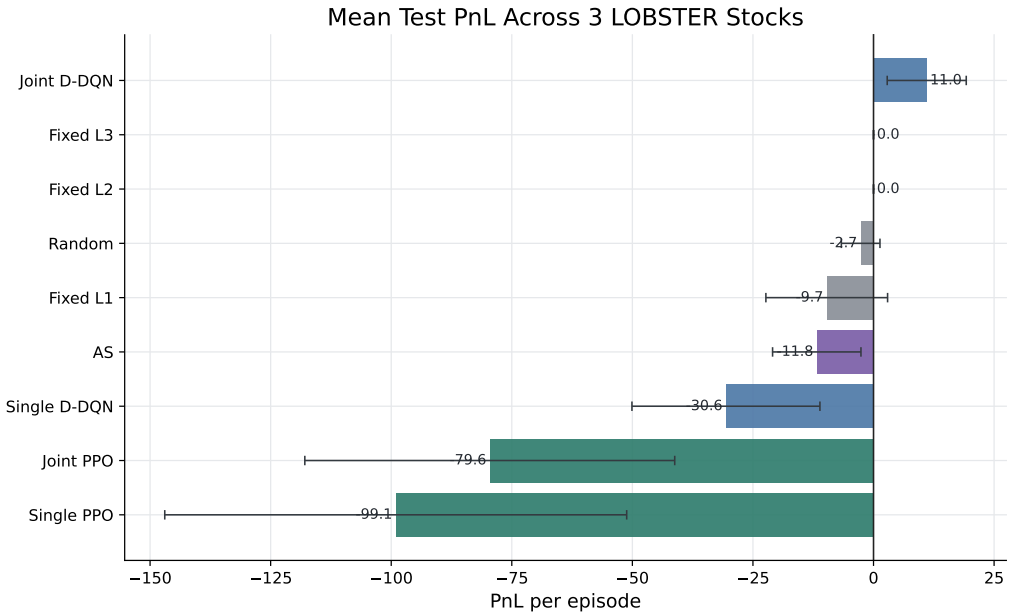


Figure 4: LOBSTER3 overall test PnL comparison.

6.4 Single vs Joint Encoder

The results support joint representation learning more clearly than they support a universal RL policy improvement. On SZ10, the joint encoder is much stronger than the median single-stock encoder in classification, and joint PPO is the best RL agent. On LOBSTER3, the joint encoder also improves classification and joint Dueling-DQN is the best trader. However, joint Dueling-DQN underperforms on SZ10, which suggests that the downstream RL objective is still sensitive to reward shape, action space, and execution details.

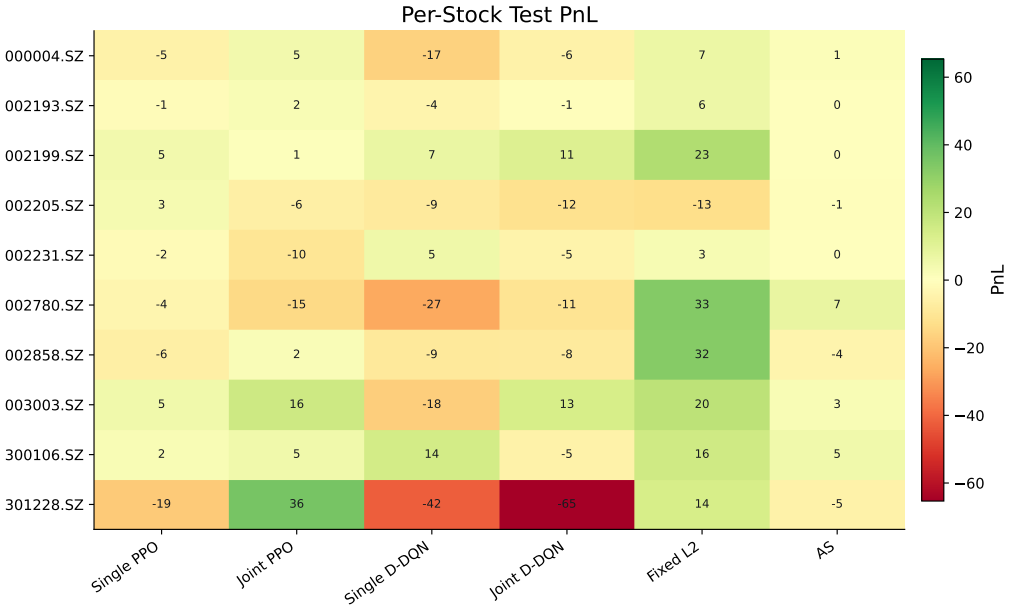


Figure 5: Per-stock SZ10 PnL heatmap. Performance varies substantially by stock.

7 Robustness Results

7.1 Latency and Order Loss

Table 6 summarizes representative latency scenarios. The SZ10 fixed-level baseline is relatively robust, while joint PPO remains positive in several observation-latency and order-loss scenarios. Joint Dueling-DQN is more fragile on SZ10 but remains the best learning agent on LOBSTER3 under several observation-latency scenarios.

Table 6: Representative latency and order-loss PnL results.

Data	Scenario	fixed_2	AS	joint PPO	joint DQN
SZ10	paper_base	13.97	0.54	3.50	-8.83
SZ10	obs_latency_5	18.45	6.14	5.87	-3.79
SZ10	obs_latency_10	2.40	6.23	-0.21	-23.90
SZ10	loss_5pct	14.00	0.76	4.42	-5.90
SZ10	infer50_loss5pct	2.27	1.84	-6.00	-20.19
LOBSTER3	paper_base	0.00	-11.77	-79.57	11.03
LOBSTER3	obs_latency_5	10.20	-18.63	-88.33	25.37
LOBSTER3	obs_latency_10	15.53	-25.63	-99.07	16.63
LOBSTER3	loss_5pct	0.00	-15.90	-75.03	14.60
LOBSTER3	infer50_loss5pct	2.60	-26.33	-29.73	-15.53

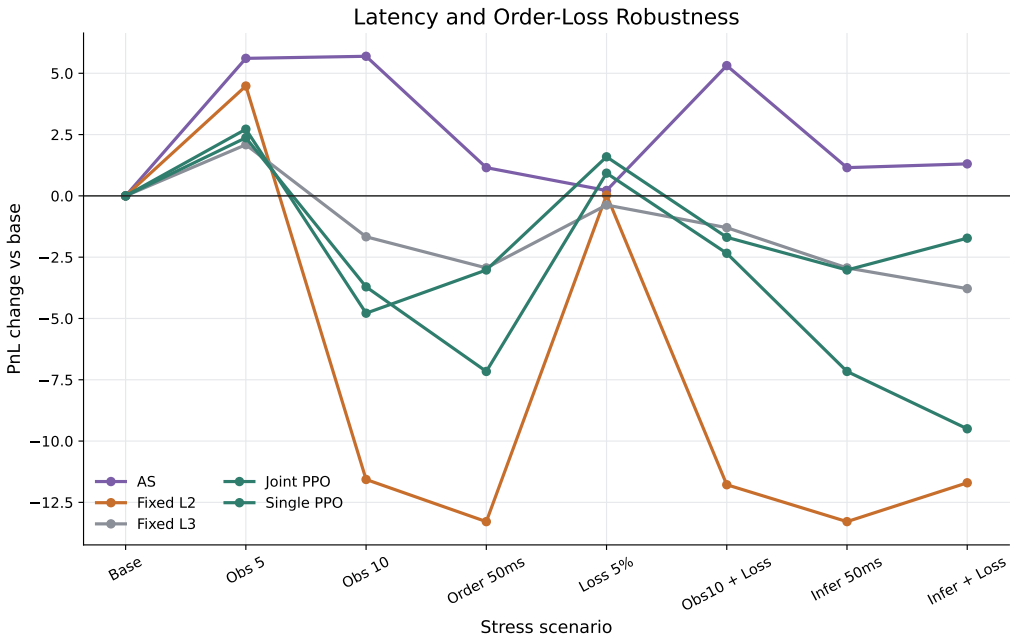


Figure 6: SZ10 PnL change under latency, order-loss, and inference-delay scenarios.

7.2 Regime Sensitivity

Regime analysis groups episodes by realized volatility, average spread, and trade intensity. For SZ10, fixed level 2 performs best in high-volatility, high-spread, and high-intensity regimes. Joint

PPO is positive in high-volatility and high-spread regimes but weaker in low-volatility episodes. For LOBSTER3, joint Dueling-DQN performs well in high-volatility and low trade-intensity episodes, while PPO is consistently negative.

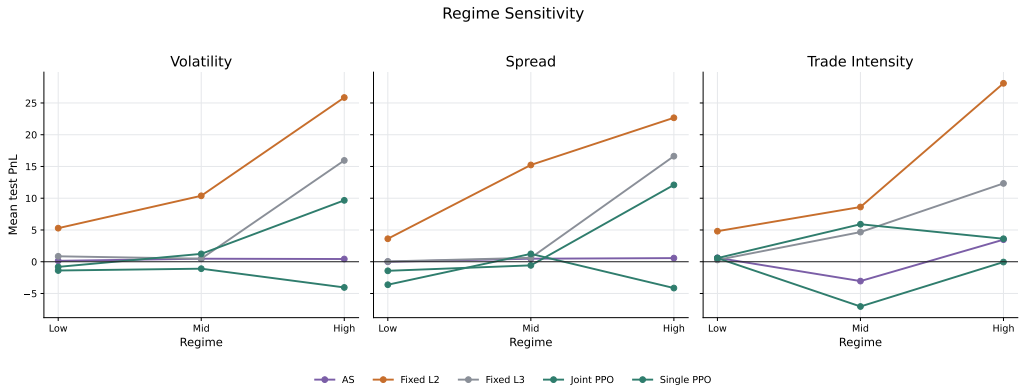


Figure 7: SZ10 regime-sensitivity results.

8 Discussion

Our main findings are:

1. **Attn-LOB pretraining is useful.** The joint Shenzhen encoder achieves strong out-of-sample classification performance, suggesting that cross-stock representation learning captures shared microstructure patterns.

2. **Better prediction does not automatically imply better market making.** Fixed-level quotes can outperform RL because the trading objective depends on inventory, fill probability, and spread capture, not only direction prediction.

3. **Baselines are essential.** Without fixed-level, random, and Avellaneda-Stoikov benchmarks, RL PnL is hard to interpret.

4. **Execution assumptions change conclusions.** Observation latency, order latency, order loss, and inference delay can change method rankings.

5. **Market and sample dependence is large.** SZ10 and LOBSTER3 lead to different best agents, so robustness across assets and days remains an open issue.

There are also limitations. The simulator does not model exact exchange queue position, partial priority, hidden liquidity, or market impact. We set explicit transaction costs to zero in the main experiments because the intended strategy is small-order market making, but real deployments would need exchange-specific fees, rebates, and latency distributions. The Shenzhen sample has only five trading dates, so the learned policies should be treated as experimental rather than production-ready.

9 Conclusion

We built a complete reproduction and extension of an Attn-LOB based market-making framework. The project now includes data reconstruction, pretraining, RL training, baseline evaluation,

latency and order-loss stress tests, regime sensitivity, and reproducible command-line interfaces. The strongest classification model is the joint SZ10 Attn-LOB encoder. The strongest SZ10 trading method is the fixed-level 2 baseline, while the strongest LOBSTER3 trading method is joint Dueling-DQN. The difference between these results is itself informative: market-making RL needs careful evaluation against simple baselines and realistic execution frictions before its PnL can be interpreted.

A Reproducibility Commands

```
cd Our-Code

# Rebuild reports and final plots from saved results
../.venv/bin/python3 run_system.py summarize

# Run the full SZ10 pipeline
../.venv/bin/python3 run_system.py full --stage all

# Run latency scenarios
../.venv/bin/python3 run_system.py latency --eval-episodes 10

# Run regime sensitivity
../.venv/bin/python3 run_system.py regime

# Run the LOBSTER3 comparison
../.venv/bin/python3 run_lobster3_full_experiment.py --stage all
```

B Output Files

The most important output files are:

- SZ10 root: `Our-Code/experiment_results/sz10_full_experiment/`
- SZ10 reports: `reports/pretrain_summary.csv`, `reports/comparison_test_aggregate.csv`
- SZ10 robustness: `latency_scenarios/reports/latency_aggregate.csv`, `regime_sensitivity/reports/regime_sensitivity_summary.csv`
- SZ10 figures: `final_plots/`
- LOBSTER3 root: `Our-Code/experiment_results/lobster3_full_experiment/`
- LOBSTER3 reports and figures: `reports/comparison_test_aggregate.csv`, `final_plots/`

References

[1] Original project paper: *Market Making with Deep Reinforcement Learning from Limit Order Books.*

[2] Avellaneda, M. and Stoikov, S. High-frequency trading in a limit order book. *Quantitative Finance*, 2008.

[3] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal Policy Optimization Algorithms. 2017.

[4] Wang, Z., Schaul, T., Hessel, M., van Hasselt, H., Lanctot, M., and de Freitas, N. Dueling Network Architectures for Deep Reinforcement Learning. 2016.